

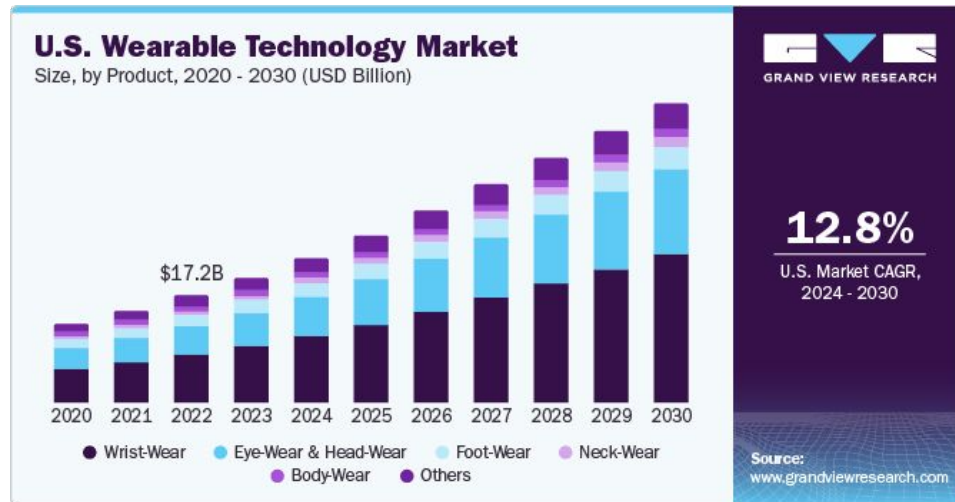


RISC-V Based AES Accelerator for Wearable Devices

**Milan Patel
Buddhi Perera
Johnson Yang**

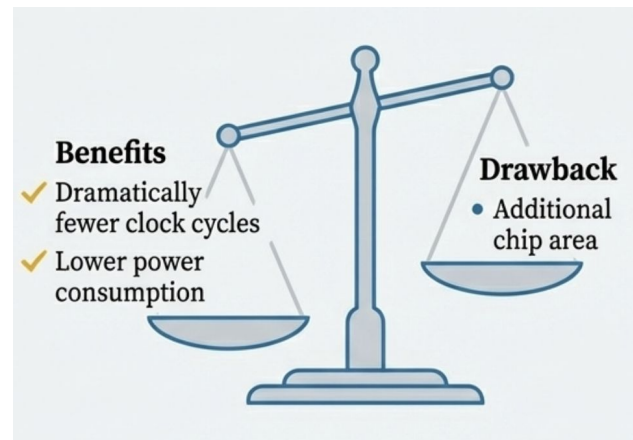
Problem and Motivation

- Wearable tech industry is rapidly growing [1]
- High data throughput devices (e.g. smart watches or medical devices)
 - Biometrics, audio, location, etc
- Need for security!
 - General purpose CPU is inefficient at encoding or decoding
 - Application specific integrated chips can be more efficient



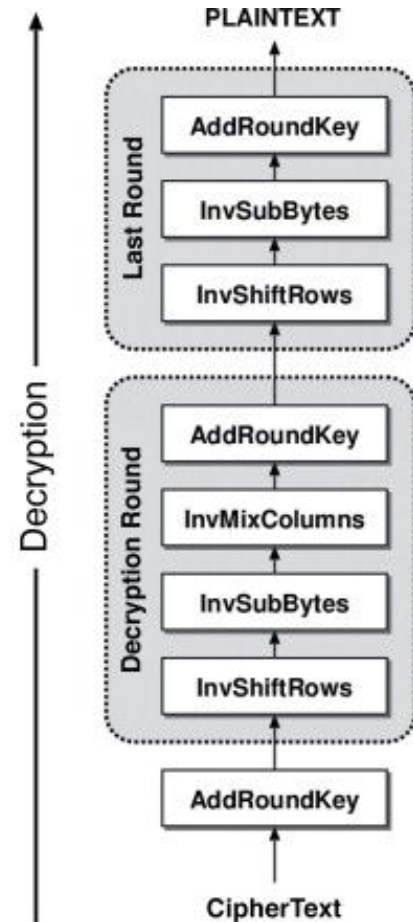
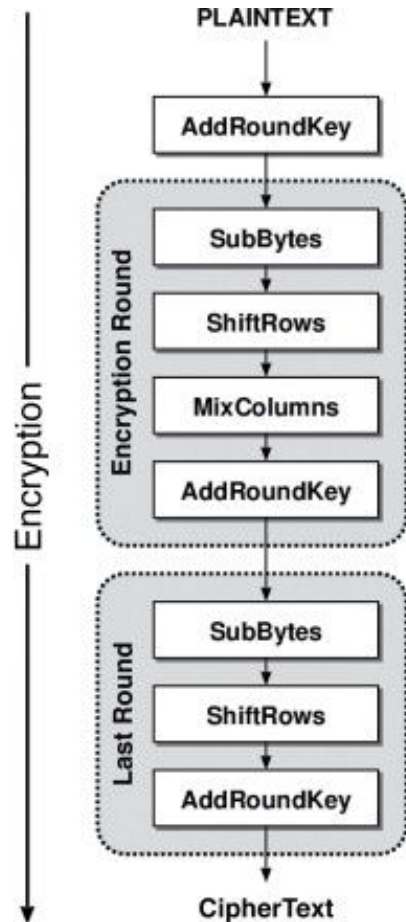
Solution - AES Accelerator

- Memory mapped AES accelerator for encryption and decryption
 - Write key and plain text/cipher text to the accelerator
 - Start the the encryption/decryption
 - Read the encrypted/decrypted values from accelerator
- Wearable devices → strict constraints on power and area.
- Benefits
 - Less clock cycles (than software implementations)
 - Less power
- Drawbacks
 - More area



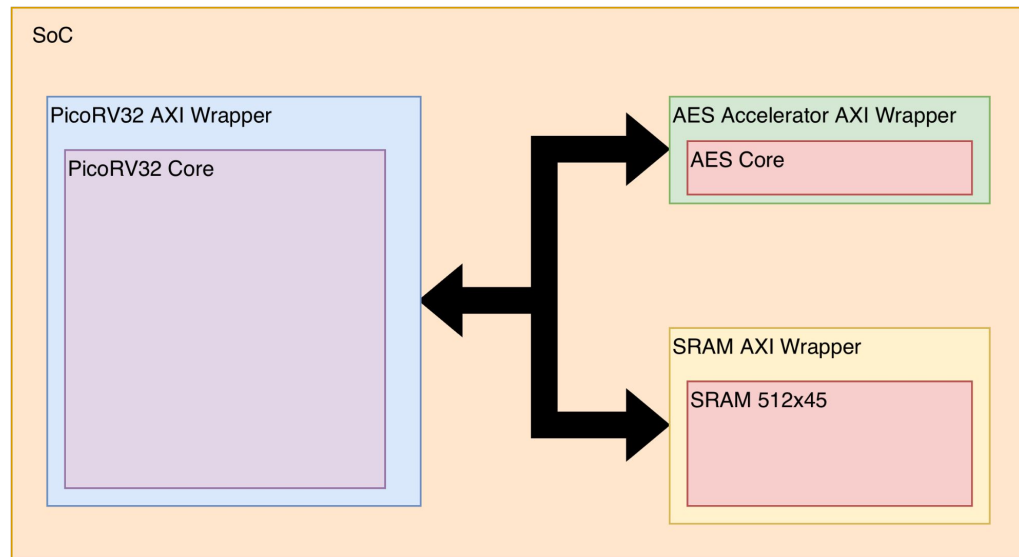
AES Algorithm

- Symmetric block cipher
- Variations (Key size)
 - AES 128 → 10 rounds → 11 keys
 - AES 192 → 12 rounds → 13 keys
 - AES 256 → 14 rounds → 15 keys
- Different modes
 - Initial seed vector
 - Feedback
 - Chaining
 - Changes in data flow
- Our Impl - AES 128 ECB mode



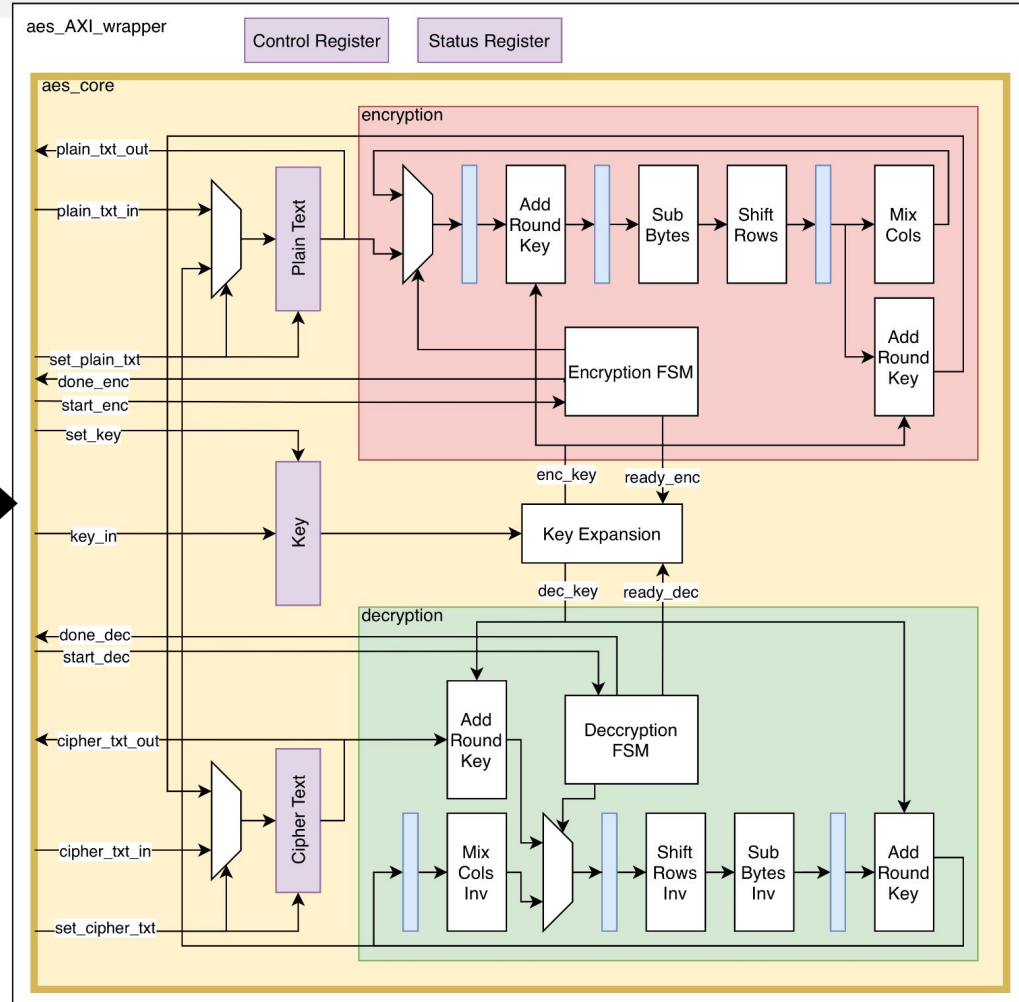
Overall Architecture

- CPU - PicoRV32
 - Open source IP by YosysHQ
 - 32 bit single stage core
 - RV32I instruction set
 - SRAM
 - TSMC SRAM macro 2kB
 - SRAM AXI Wrapper in SystemVerilog
 - AES Accelerator
 - Written in SystemVerilog
 - Interconnect
 - AXI4 Lite interface
- NYU Memory mapped IO



AES Accelerator

- 3 cycles per round - 30 cycles/block
- Round keys generated on the fly in encryption
- Round keys are stored for use in decryption



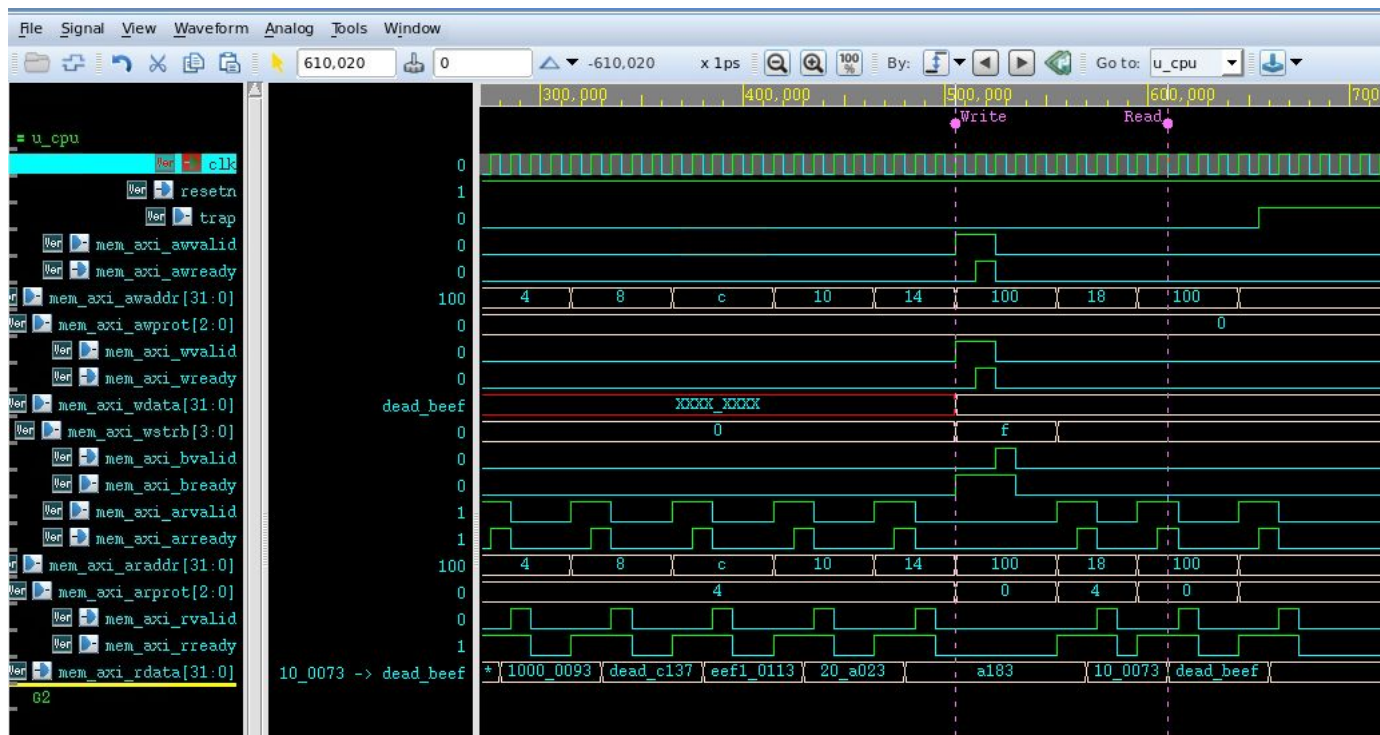
Verification - AES

- RTL Waveform simulations
- Test vectors
 - Known Test Vectors - NIST Standard
 - Online AES tools



```
rtl
├── aes
│   ├── pkgs
│   └── tb
│       ├── tb_add_roundkey.sv
│       ├── tb_aes_core.sv
│       ├── tb_decryption.sv
│       ├── tb_encryption_fsm.sv
│       ├── tb_encryption.sv
│       ├── tb_key_expansion.sv
│       ├── tb_mix_cols.sv
│       ├── tb_pipeline_reg.sv
│       ├── tb_shift_rows.sv
│       ├── tb_sub_bytes_inv.sv
│       ├── tb_sub_bytes.sv
│       └── tb_top_aes.sv
├── add_roundkey.sv
├── aes_core.sv
├── decryption_fsm.sv
├── decryption.sv
├── dff.sv
├── encryption_fsm.sv
├── encryption.sv
├── key_expansion.sv
├── Makefile
├── mix_cols.sv
├── mux.sv
├── pipeline_reg.sv
├── s_box_inv.sv
├── s_box.sv
├── shift_rows.sv
├── sub_bytes_inv.sv
├── sub_bytes.sv
├── tb_sub_bytes_inv.fsd
└── top_aes.sv
```

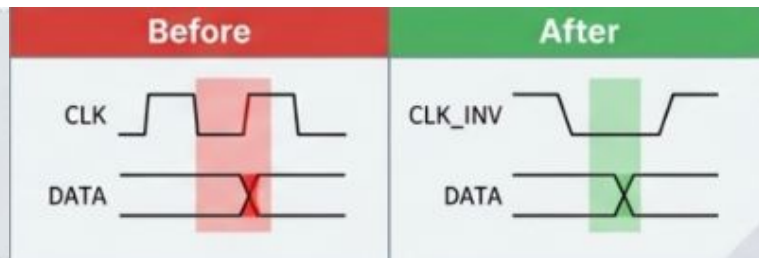
Verification - CPU & SRAM via AXI



Challenges - SRAM

- The SRAM Verilog model triggered **timing assertions** and **error messages** verilog model for **timing violations**, as the data signals were changing too close to the sampling edge during the half-cycle access.
- **Fix:** Implemented an **inverted clock** for the SRAM to strictly define the phase relationship between the CPU and memory, ensuring the signals remained stable long enough to satisfy the model's **hold-time requirements**.

```
"N16ADFP_SRAM_100a.v", 14919: Timing violation  
in tb_axi_sram.u_dut.u_sram_512x45  
$hold( posedge CLK &&& check_write:115000,  
posedge BWEB[17]:115000, limit: 1 );
```



Verification - SoC

Encryption

```
# loading the key from the SRAM and store in the AES key register
lw    x2, 0(x4)      # 00022103
sw    x2, 16(x3)     # 0021a823
lw    x2, 4(x4)      # 00422103
sw    x2, 20(x3)     # 0021aa23
lw    x2, 8(x4)      # 00822103
sw    x2, 24(x3)     # 0021ac23
lw    x2, 12(x4)     # 00c22103
sw    x2, 28(x3)     # 0021ae23

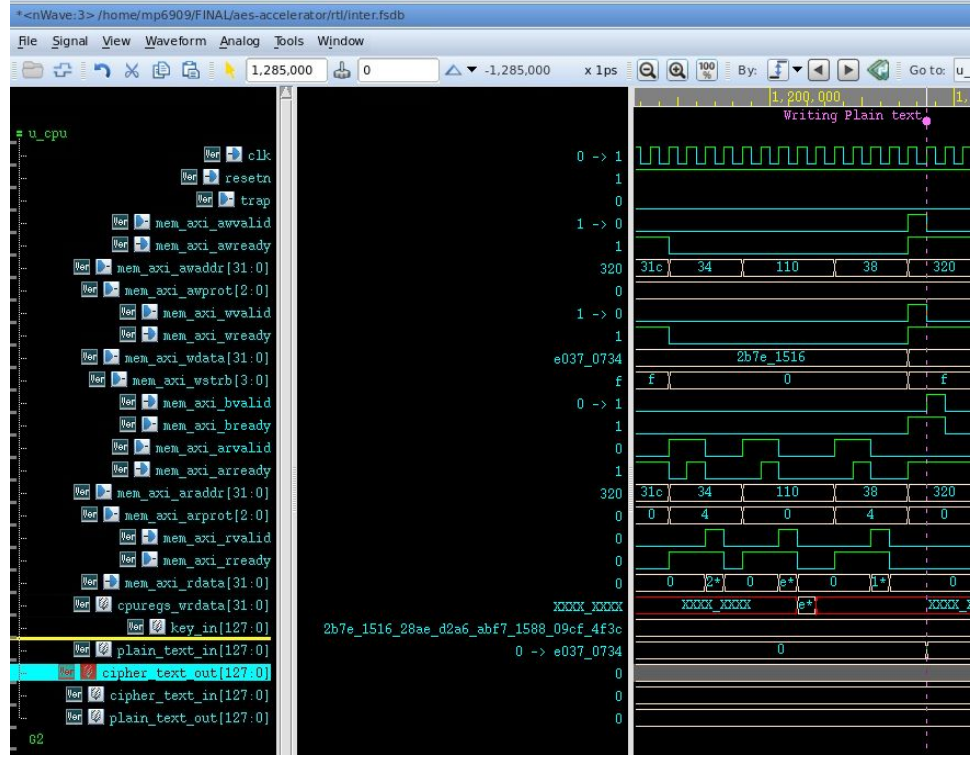
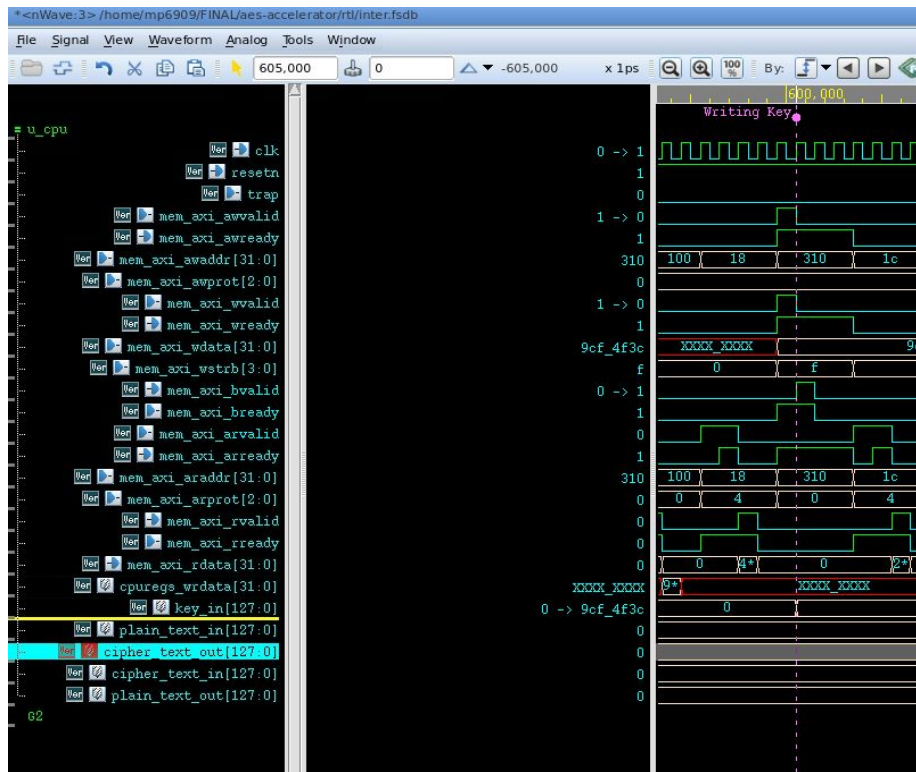
# loading the plain text from the SRAM and store in the AES plain text register
lw    x2, 16(x4)     # 01022103
sw    x2, 32(x3)     # 0221a023
lw    x2, 20(x4)     # 01422103
sw    x2, 36(x3)     # 0221a223
lw    x2, 24(x4)     # 01822103
sw    x2, 40(x3)     # 0221a423
lw    x2, 28(x4)     # 01c22103
sw    x2, 44(x3)     # 0221a623

# starting the encryption
addi  x2, x0, 1      # 00100113
sw    x2, 0(x3)      # 0021a023

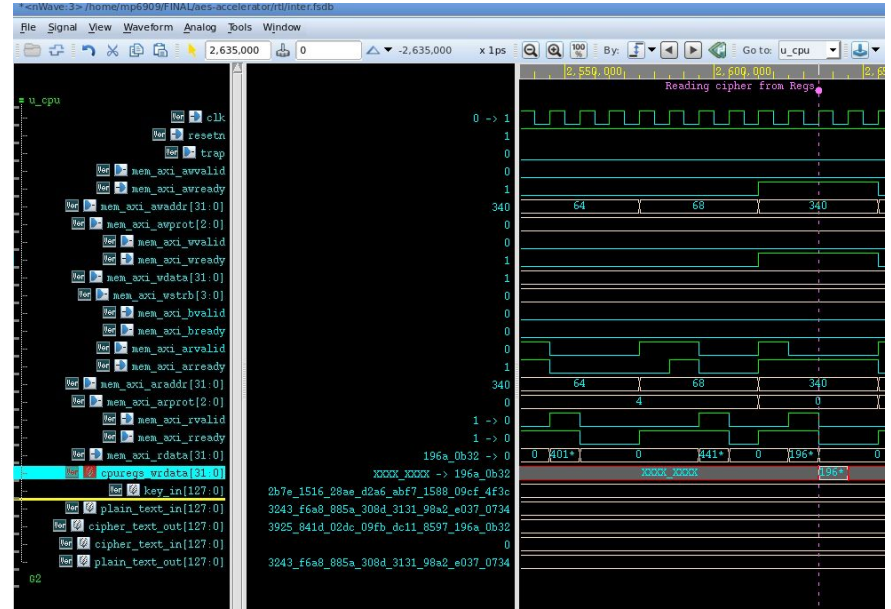
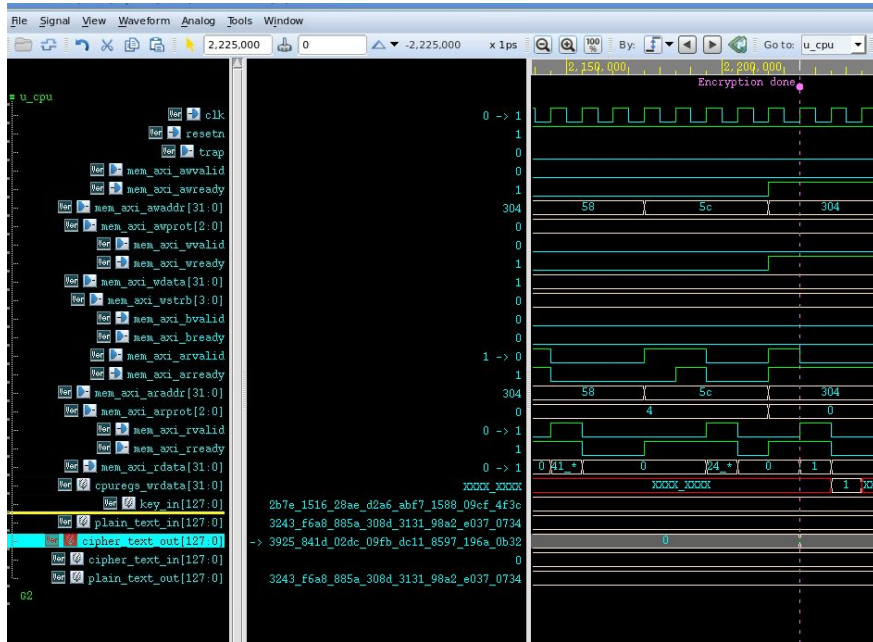
# polling for the done status
lb    x9, 4(x3)      # 00418483
andi  x9, x9, 2      # 0024f493
beq   x9, x0, -8      # fe048ce3    (branch backward two instructions)

# read the cipher text from AES cipher text register
lw    x10, 64(x3)    # 0401a503
lw    x11, 68(x3)    # 0441a583
lw    x12, 72(x3)    # 0481a603
lw    x13, 76(x3)    # 04c1a683
```

Verification - Writing Key and Plain text



Verification - Encryption



Verification - SoC

Decryption

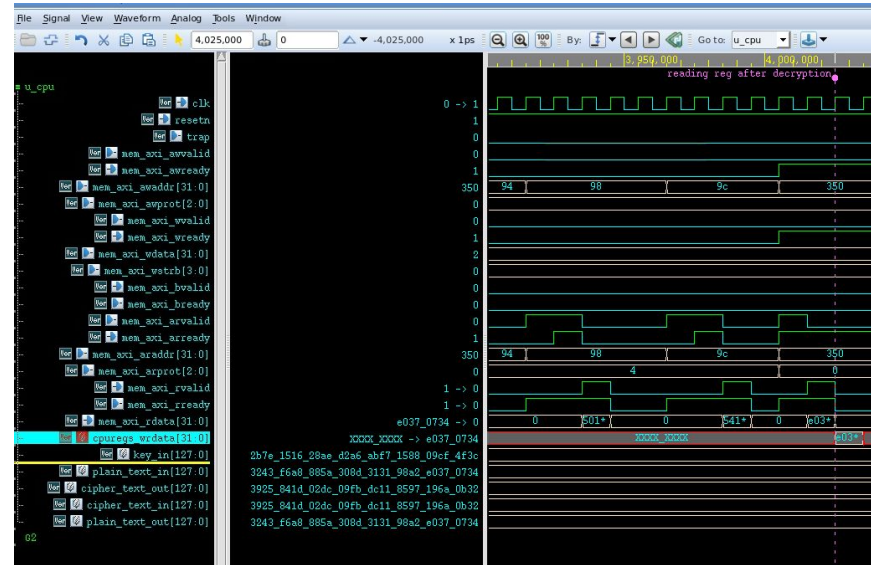
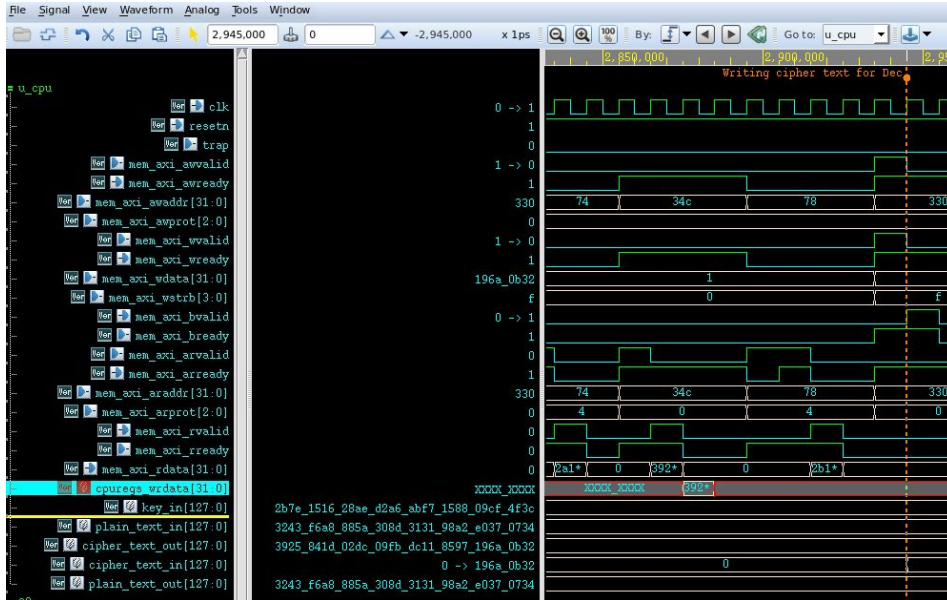
```
# store the cipher text in the AES cipher text register
sw    x10, 48(x3)      # 02a1a823
sw    x11, 52(x3)      # 02b1aa23
sw    x12, 56(x3)      # 02c1ac23
sw    x13, 60(x3)      # 02d1ae23

# starting the decryption
addi   x2, x0, 2        # 00200113
sw     x2, 0(x3)         # 0021a023

# polling for the done status
lb     x9, 4(x3)         # 00418483
andi   x9, x9, 8         # 0084f493
beq     x9, x0, -8        # fe048ce3 (branch backward two instructions)

# read the plain text from AES cipher text register
lw     x10, 80(x3)       # 0501a503
lw     x11, 84(x3)       # 0541a583
lw     x12, 88(x3)       # 0581a603
lw     x13, 92(x3)       # 05c1a683
```

Verification - Top Module (Decryption Waveform)



Evaluation – Targets

- Chip Area: 4.5mm²
 - 3.8mm² (RISC-V CPU)
 - 0.2mm² (AES Accelerator)
 - 0.5mm² (SRAM)
- Average Power: 5mW [1]
- Cycles to Encrypt Plain Text: 340 for key expansion & 912 for encryption of 16-byte block [2]
 - Benchmarked with RISC-V 32-bit 5-stage pipelined CPU
 - Using optimizations in software based on [3]

Evaluation – Results

Input Activity:

Dominant Frequency: (MHz)

Flop Activity:

Clock Gate Activity:

Total Power

Total Internal Power:

1.83363742

65.5681%

Total Switching Power:

0.90390089

32.3221%

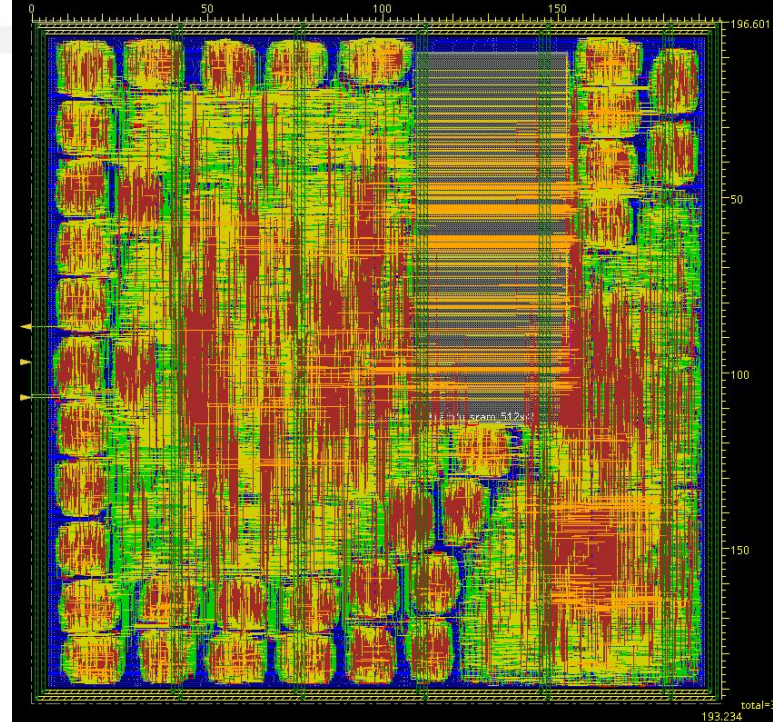
Total Leakage Power:

0.05900075

2.1098%

Total Power:

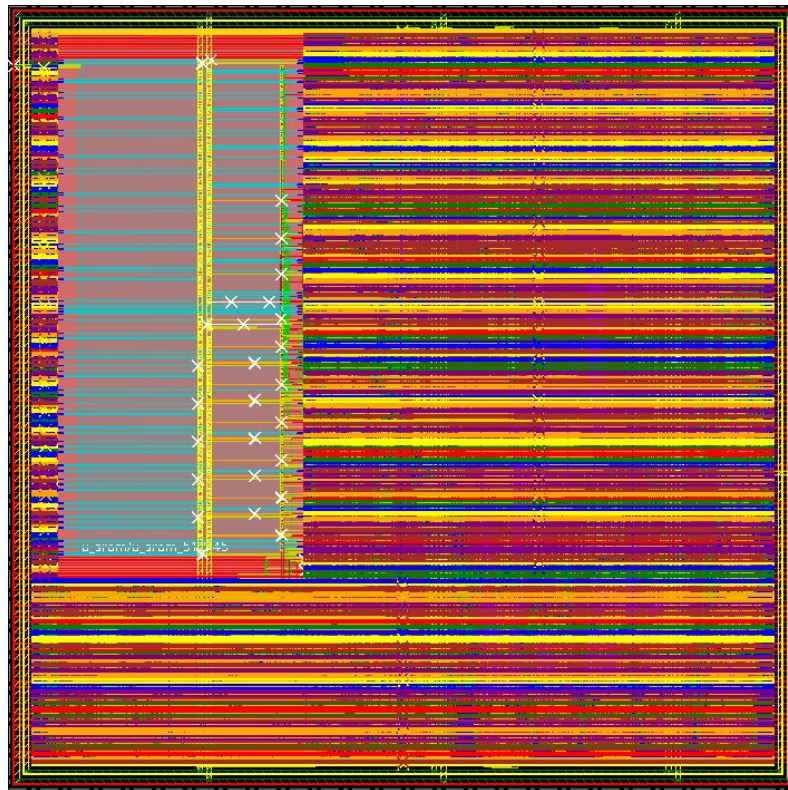
2.79653906



Metric	Target	Our Results
Chip Area	4.5mm ²	0.04mm²
Average Power	5mW	2.797mW
Cycles (expansion & encryption)	1252	168 (30 +138 transfer overhead)

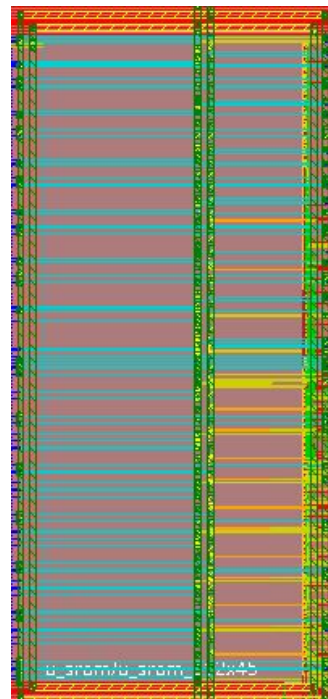
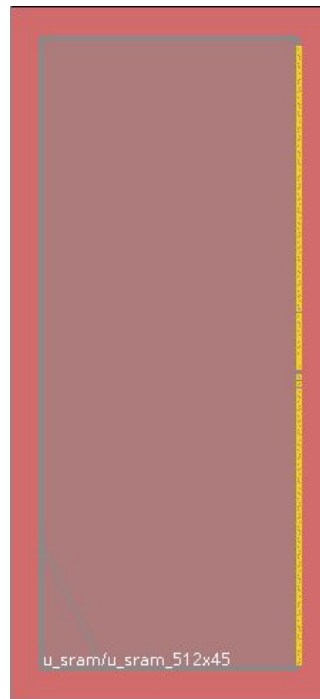
Challenges – LVS

- Passed DRC & STA but...
thousands of LVS errors
- What We Tried?
 - Alternate SRoute setups
 - Testing multiple global net connections
 - Larger floorplan → More space for Vias/Wires
 - Fillers → cover gaps to fill in open violations



Challenges - Macros

- Proper Connection to Macros
- What We Tried
 - Plain
 - Boundaries
 - Rings
- Takeaways
 - Chip design is heavily iterative, as well as a trial and error process!



Use of Generative AI in the Design Flow

What Worked?

- Survey and analyze the possible design techniques
 - Memory mapped vs tightly coupled ISA extension
- Developing RTL code
 - AXI interfaces
 - AES submodules
 - Comprehensive testbenches

What Didn't?

- Not as helpful in physical design
 - Nonexistent commands or wrong syntax

Conclusion

- Takeaways
 - Our hardware based solution outperformed the software implementations
 - Necessity of open-source material (PicoRV-32)
 - Agile works!
- Experiences
 - Learning how to design hardware for accelerators
 - Interconnecting peripherals in a SoC (Memory mapping)
 - Hands on experience with EDA tools (Genus, Innovus, Calibre, etc)
- Future work
 - Fix LVS in the current design associated with the macros
 - Configurable for different key sizes (192 and 256 key sizes)
 - Support other AES modes

Q&A